# ТЕМА 3. КАСКАДНЫЕ ТАБЛИЦЫ СТИЛЕЙ

## ЗАДАНИЕ 3.8. СПЕЦИФИЧНОСТЬ СТИЛЕЙ.

Еще одно не совсем традиционное задание. Внимательно изучите представленный теоретический материал и примеры кода. Вносите изменения, экспериментируйте, исследуйте!

**Введение**

С увеличением вашей базы знаний в части html-элементов и css-свойств сложность кода, равно как и его количество, будет расти.

Начинающие разработчики очень часто не рассматривают верстку и стилизацию проекта как единое целое, ограничиваясь лишь оформлением сначала одного элемента, затем другого, потом третьего. Все это приводит к избыточности кода, дублированию свойств и неразберихе в итоговом проекте.

Внешне в браузере все выглядит более-менее прилично, но вот в редакторе… Такая ситуация приводит к тому, что внесение любых последующих изменений в код приводит к неожиданным последствиям. Съезжают блоки, не меняются настройки цвета или текста, пропадают или появляются из ниоткуда свойства элементов.

Одним из способов избежать таких проблем является не быстрый переход к крутым навороченным проектам, а **последовательное изучение и понимание основ разметки**: DOM-дерево, наследование, позиционирование элементов, специфичность стилей… На последнем остановимся подробнее.

**Общая информация**

Специфичность - это способ, с помощью которого браузеры определяют, какие значения свойств CSS наиболее соответствуют элементу и, следовательно, будут применены.

Значение специфичность имеет в случае конфликта, когда одному элементу на странице назначено более одного значения какого-либо свойства.


©Pechora-PRO

Специфичность представляет собой вес, придаваемый конкретному правилу CSS. Вес правила определяется количеством каждого из типов селекторов в данном правиле.

Согласно спецификации CSS, правило для непосредственно соответствующего элемента всегда будет иметь больший приоритет, чем правила, унаследованные от предка.

Например, у нас есть верстка и стили (см. sample-1):

| html | css | результат |
|---|---|---|
| <br><br> ... <br><br> ... <br><br> | div {<br>color:red;<br>}<br><br>p {<br>color:blue;<br>} | Для содержимого элемента **span** не установлены стили, поэтому цвет текста получен от родителя - **div**.<br><br>Для абзаца **p** - определен свой собственный цвет, именно он и будет применен к элементу. |

### Типы селекторов

В следующем списке типы селекторов расположены **по возрастанию** специфичности:

- селекторы типов элементов (например, h1) и псевдоэлементов*****;
- селекторы классов (например, .example), селекторы атрибутов (например, [type="email"]) и псевдоклассов*****;
- селекторы идентификаторов (например, #example).

***Псевдоэлементы и псевдоклассы рассмотрим в одном из следующих уроков.**

Если у нескольких правил специфичность одинакова, то к элементу применяется последнее по порядку правило CSS.

Рассмотрим следующие сочетания стилей и селекторов (см. sample-2):


©Pechora-PRO

| html | css | результат |
|---|---|---|
|  ... <br><br> ... <br><br> ... <br><br><p class="select"<br>id="text"> ...  | p { color: blue; }<br><br>.select { color: red; }<br><br>#text { color: green; } | Класс **select** переопределит настройки для **p**, также как и идентификатор **text**.<br><br>При этом если класс и идентификатор указать в одном элементе, то «победит» идентификатор |
| <span<br>class="first second"><br>...<br> | .first {<br>font-size: xx-large;<br>}<br>.second {<br>font-size: small;<br>} | Для элемента **span** заданы два класса.<br><br>Размер текста будет зависеть от порядка записи стилей в **css**. |

Стили, объявленные в элементе (например, style="font-weight:bold"), всегда переопределяют любые правила из внешних файлов стилей и, таким образом, их специфичность можно считать наивысшей. Мы рассматривали такой вариант, когда говорили о наследовании в предыдущей лекции.

Взаимное расположение элементов, указанных в селекторе не влияет на специфичность правила.

Универсальный селектор (*), комбинаторы (+, >, ~, ' ') и отрицающий псевдокласс (:not()) не влияют на специфичность (однако селекторы, объявленные внутри :not(), влияют).

### Модификатор !important

Когда при объявлении стиля используется модификатор **!important**, это объявление получает **наивысший приоритет** среди всех прочих объявлений.

Если к элементу применимы два взаимоисключающих стиля с модификатором **!important**, то применён будет стиль с большей специфичностью.


©Pechora-PRO

Использовать **!important** допустимо в страничных стилях, которые переопределяют стили сайта или внешние стили (полученные из библиотек) или в самом крайнем случае!

Помните, что отследить такое «перекрытие» в свойствах элемента потом будет достаточно сложно.

Старайтесь обойтись без модификатора, используя свойства с высокой специфичностью, например, добавьте указания на родителей нужного вам элемента – специфичность повысится и ваше свойство отработает.

Отрицающий псевдокласс **:not** не учитывается как псевдокласс при расчёте специфичности. Однако селекторы, расположенные внутри **:not**, при подсчёте количества по типам селекторов рассматриваются как обычные селекторы и учитываются.

**Важно!** Некоторые моменты в лекции мы еще не рассматривали (например, комбинаторы, псевдоклассы и псевдоэлементы), поэтому предлагаю вернуться к этому уроку еще раз ближе к окончанию курса и взглянуть на вопросы специфичности с позиции большего опыта в верстке, создании стилей и использования селекторов.


©Pechora-PRO